

Anàlisi de Rendiment en Problemes de Geometria Computacional: Comparació d'Estratègies per a la Cerca de Parelles de Punts en 2D

Josep Oliver Vallespir, Hugo Valls Sabater

Resum—Aquest article presenta el desenvolupament i l'anàlisi de diferents algorismes per resoldre dos problemes clàssics de la geometria computacional: la cerca de la parella de punts més propera i la parella més llunyana en un espai bidimensional. S'han implementat dues estratègies principals per al càlcul de la distància mínima: una aproximació de força bruta amb complexitat $O(n^2)$ i una altra basada en el paradigma de Divideix i Venceràs amb complexitat $O(n \log n)$. De manera complementària, s'ha desenvolupat un algorisme per determinar la distància màxima entre punts. El sistema s'ha implementat en Java seguint el patró Model-Vista-Controlador (MVC) i inclou una interfície gràfica que permet visualitzar els resultats. A més, s'han realitzat experiments amb diferents distribucions de punts per analitzar l'impacte en el rendiment dels algorismes.

Index Terms—Geometria computacional, Divideix i Venceràs, complexitat algorísmica, MVC, Java, núvol de punts.

1 Introducció

El càlcul de distàncies entre punts en un espai bidimensional constitueix un dels problemes fonamentals dins de la geometria computacional. En particular, la determinació de la parella de punts més propera (Closest Pair Problem) és un problema clàssic que posa de manifest la importància de dissenyar algorismes eficients capaços de gestionar grans volums de dades.

En aquesta pràctica s'ha desenvolupat una aplicació capaç de generar un núvol de punts en un pla 2D i calcular tant la parella de punts més propera com la més llunyana. Per resoldre el problema de la distància mínima s'han implementat dues estratègies diferenciades: una aproximació basada en força bruta amb complexitat $O(n^2)$ i una altra basada en el paradigma de Divideix i Venceràs, que redueix la complexitat a $O(n \log n)$.

A més, el sistema permet generar punts utilitzant diferents distribucions estadístiques, com la uniforme, la gaussiana i l'exponencial, fet que permet analitzar com la distribució espacial influeix en el rendiment dels algorismes. L'aplicació incorpora una interfície gràfica desenvolupada amb Java Swing que facilita la visualització dels resultats de manera intuïtiva.

2 Estructura del Programa (MVC)

Amb l'objectiu de mantenir una arquitectura clara, escalable i fàcil de mantenir, l'aplicació s'ha desenvolupat seguint el patró de disseny Model-Vista-Controlador

(MVC). Aquest patró permet separar de manera clara les responsabilitats del sistema, evitant la barreja de lògica, dades i interfície gràfica.

A la Figura 1 es pot observar l'estructura real del projecte, organitzada en diferents paquets segons la seva funcionalitat.

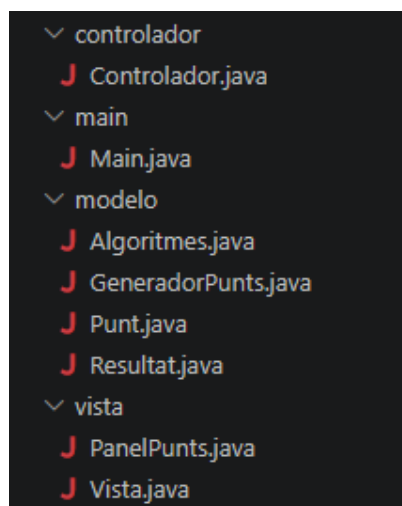


Figura 1. Estructura de packages del projecte.

Tal com es pot veure, el projecte es divideix en tres paquets principals: modelo, vista i controlador, a més del paquet main, que actua com a punt d'entrada del programa.

El paquet modelo constitueix el nucli matemàtic del sistema. En aquest es defineixen les estructures de dades i els algorismes necessaris per resoldre el problema. Classes com Punt, Resultat i Algoritmes encapsulen

- *Estudiants del Grau en Enginyeria Informàtica a la Universitat de les Illes Balears (UIB).*
Assignatura: Algorismes Avançats.

tota la lògica relacionada amb el càlcul de distàncies, mantenint-se completament independents de qualsevol element gràfic.

Per altra banda, el paquet vista conté totes les classes relacionades amb la interfície gràfica. La classe Vista defineix la finestra principal i els elements d'interacció amb l'usuari, mentre que PanelPunts s'encarrega de la representació visual dels punts i dels resultats. Aquesta separació permet modificar la interfície sense afectar la lògica del programa.

El paquet controlador actua com a pont entre la vista i el model. La classe Controlador rep les accions de l'usuari, interpreta les opcions seleccionades i invoca els mètodes corresponents del model. Posteriorment, retorna els resultats a la vista perquè siguin representats gràficament.

Finalment, el paquet main conté la classe d'inicialització del programa. Aquesta és l'encarregada de crear la vista i el controlador, establint la connexió entre ambdós components i posant en marxa l'aplicació.

Aquesta organització basada en MVC no només facilita la comprensió del codi, sinó que també permet escalar el sistema de manera senzilla. Per exemple, és possible afegir nous algorismes o noves formes de visualització sense necessitat de modificar la resta de components, fet que demostra la robustesa del disseny adoptat.

3 La Vista

Seguint el patró MVC, la vista s'encarrega exclusivament de la representació gràfica i de la interacció amb l'usuari, mantenint-se completament independent de la lògica de càlcul. Aquesta separació ha estat especialment important en aquesta pràctica, ja que permet modificar la manera de visualitzar els punts sense afectar en cap moment els algorismes implementats.

La interfície s'ha desenvolupat utilitzant Java Swing, amb l'objectiu de crear un entorn visual clar, intuïtiu i funcional. La vista es divideix principalment en dues classes: Vista, que defineix la finestra principal, i PanelPunts, que actua com a llenç gràfic.

3.1 La finestra principal

La classe Vista defineix l'estructura general de l'aplicació. Aquesta es construeix utilitzant un gestor de disposició BorderLayout, que permet dividir la pantalla en diferents regions clarament diferenciades.

A la part esquerra de la finestra es troba el panell de configuració, on l'usuari pot seleccionar el nombre de punts a generar, així com el tipus de distribució (uniforme o gaussiana). També s'hi inclouen les opcions per escollir l'algorisme a utilitzar, permetent comparar directament el comportament de les diferents estratègies.

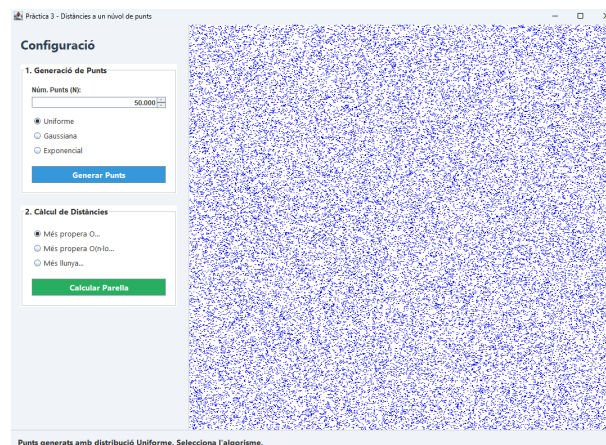


Figura 2. Generació de punts uniforme.

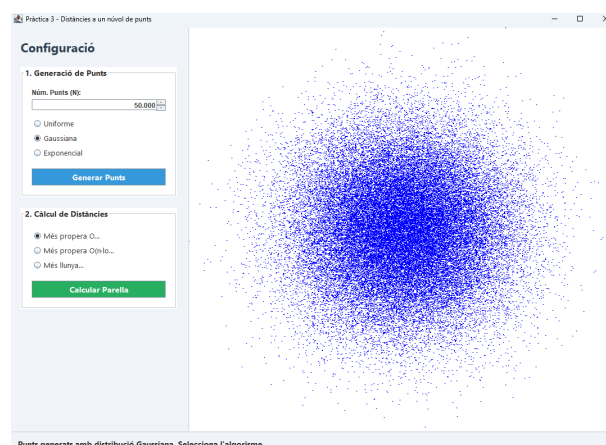


Figura 3. Generació de punts gaussiana.

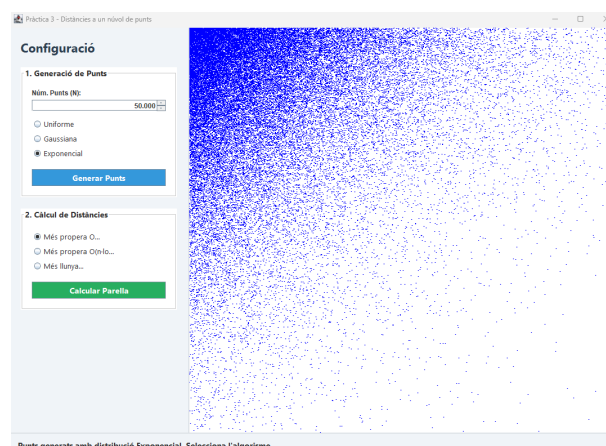


Figura 4. Generació de punts exponencial.

A la part inferior es situa una barra d'estat que mostra informació rellevant durant l'execució, com ara el resultat obtingut, les coordenades dels punts seleccionats i el temps de càlcul. Aquest element és especialment útil per proporcionar feedback immediat a l'usuari.

Finalment, la zona central de la finestra està reservada per a la representació gràfica dels punts, la qual es delega a la classe `PanelPunts`.

A més, s'ha incorporat un botó que permet pausar l'execució dels algorismes en qualsevol moment. D'aquesta manera, l'usuari pot interrompre temporalment el procés,

3.2 El panell de dibuix

El component `PanelPunts` és el responsable de representar visualment el núvol de punts i els resultats dels algorismes. En lloc d'utilitzar components gràfics individuals per a cada punt, s'ha optat per dibuixar-los manualment mitjançant la sobreescritura del mètode `paintComponent(Graphics g)`.

Aquest enfocament presenta diversos avantatges. En primer lloc, permet adaptar dinàmicament la representació a la mida de la finestra, fent que els punts es redistribueixin correctament en cas de redimensionament. En segon lloc, evita el cost computacional associat a la gestió de múltiples components gràfics, millorant així el rendiment global de l'aplicació.

Per representar els punts, es realitza una transformació de les coordenades normalitzades $(0, 1)$ a coordenades de pantalla en píxels. A més, s'ha tingut en compte el centrament dels punts, de manera que la seva posició representada coincideixi exactament amb la coordenada matemàtica i no amb una cantonada del dibuix.

Quan es calcula la parella de punts més propera o més llunyana, aquests es destaquen mitjançant una línia vermella que els uneix. En el cas de la distància mínima, s'ha afegit un element visual addicional: una el·lipse orientada segons la direcció dels punts. Aquesta decisió de disseny s'ha pres per millorar la visibilitat en situacions en què els punts es troben molt propers entre si, ja que la línia per si sola pot resultar difícil de percebre.

Durant el desenvolupament, es van identificar diverses dificultats relacionades amb la representació gràfica, com ara el desplaçament incorrecte dels punts o la persistència de resultats anteriors en pantalla. Aquestes incidències es van resoldre ajustant el centrament dels dibuixos i gestionant correctament l'estat del panell, assegurant que cada nova execució refresqui completament la visualització.

En conjunt, la implementació de la vista permet una representació clara i eficient del problema, facilitant

la comprensió visual del comportament dels diferents algorismes.

Finalment, la representació dels resultats s'ha dissenyat amb l'objectiu de facilitar la seva interpretació visual, combinant informació gràfica i numèrica.

Quan es calcula la parella de punts més propera o més llunyana, aquests es destaquen visualment dins del núvol de punts mitjançant una línia vermella que els connecta. En el cas de la distància mínima, s'hi afegeix també una el·lipse orientada segons la direcció dels punts, fet que permet identificar amb més claredat la zona on es troba la solució.

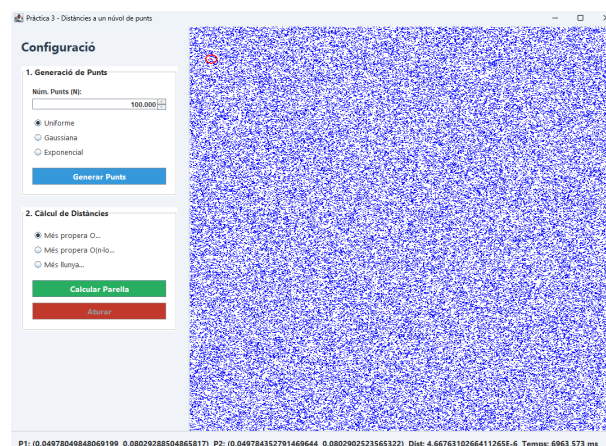


Figura 5. Visualització completa del resultat sobre el núvol de punts.

Per tal de millorar la comprensió, es pot observar amb més detall la zona on es troba la parella de punts seleccionada, destacada en vermell.

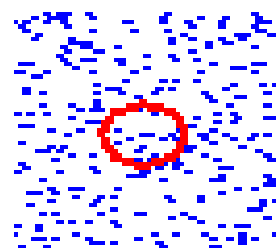


Figura 6. Detall de la parella de punts més propera ressaltada.

A més de la representació gràfica, la informació numèrica del resultat es mostra a la barra d'estat situada a la part inferior de la finestra. Aquesta inclou les coordenades dels punts, la distància calculada i el temps d'execució de l'algorisme.

P1: (0.3890081113995961, 0.5589965411694521) P2: (0.38901715581614205, 0.5589980224921303) Dist: 9.164921578118846-6 Temps: 1738.353 ms

Figura 7. Informació numèrica del resultat mostrada a la interfície.

D'aquesta manera, es proporciona una representació completa del resultat, combinant la intuïció visual amb dades quantitatives que permeten analitzar el comportament dels algorismes amb precisió.

4 El Model

Dins de l'arquitectura MVC, el model representa el nucli lògic i matemàtic de l'aplicació. La seva funció principal és encapsular totes les dades i els algorismes necessaris per resoldre el problema, mantenint-se completament independent de qualsevol element relacionat amb la interfície gràfica.

Aquesta separació ha estat una decisió clau durant el desenvolupament, ja que permet treballar amb la lògica del problema de manera aïllada, sense haver de tenir en compte aspectes visuals com píxels o components gràfics. D'aquesta manera, el model es converteix en una capa reutilitzable i fàcilment extensible.

4.1 Representació dels punts

La classe `Punt` constitueix l'element fonamental del sistema, representant un punt en l'espai bidimensional mitjançant coordenades (x, y) . Aquestes coordenades es generen en un rang normalitzat entre 0 i 1, fet que simplifica tant el càlcul matemàtic com la seva posterior representació gràfica.

A més, aquesta classe inclou el càlcul de la distància euclidiana entre dos punts, utilitzant la fórmula:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Aquesta operació és la base de tots els algorismes implementats i es crida de manera intensiva durant l'execució.

4.2 Generació de punts

La classe `GeneradorPunts` s'encarrega de crear el conjunt de punts inicial. Aquesta classe permet generar punts seguint diferents distribucions estadístiques, com la uniforme, la gaussiana i l'exponencial, oferint així la possibilitat d'analitzar com la distribució espacial afecta el comportament dels algorismes.

Aquesta decisió de disseny no només aporta flexibilitat al sistema, sinó que també permet realitzar experiments més complets, comparant situacions en què els punts estan distribuïts de manera homogènia amb altres en què es concentren en determinades regions.

4.3 Representació dels resultats

La classe `Resultat` s'utilitza per encapsular la informació retornada pels algorismes. Aquesta conté els dos punts que formen la parella i la distància entre ells.

Aquesta abstracció permet simplificar la comunicació entre el model i el controlador, evitant la necessitat de retornar múltiples valors de manera separada i facilitant el manteniment del codi.

4.4 Implementació dels algorismes

La classe `Algoritmes` constitueix el component central del model, ja que conté la implementació de totes les estratègies utilitzades per resoldre el problema.

Un aspecte important del disseny és que tots els algorismes treballen sobre la mateixa estructura de dades (`List<Punt>`), fet que permet comparar-los directament sense necessitat de transformacions addicionals. Aquesta decisió simplifica el codi i facilita la integració amb el controlador.

A més, la implementació s'ha realitzat de manera modular, separant clarament cada estratègia en mètodes independents. Això permet afegir nous algorismes en el futur sense modificar la resta del sistema.

En conjunt, el model es configura com una capa robusta i eficient, encarregada exclusivament de la lògica del problema, i totalment desacoblada de la interfície gràfica.

5 El Controlador

El controlador és l'element encarregat de coordinar la interacció entre la vista i el model, actuant com a intermediari entre ambdós components. Dins del patró MVC, aquesta capa és la responsable de gestionar el flux del programa i assegurar que cada acció de l'usuari es tradueixi en una operació concreta sobre les dades.

La classe `controlador` centralitza aquesta funcionalitat, gestionant els esdeveniments generats a la interfície gràfica mitjançant `ActionListener`. Quan l'usuari interactua amb la vista, per exemple prement el botó de generar punts o el de calcular distàncies, el controlador rep aquesta acció i inicia el procés corresponent.

El funcionament general del sistema es pot descriure com una seqüència clara d'operacions. En primer lloc, el controlador obté els paràmetres seleccionats a la vista, com el nombre de punts o el tipus de distribució. A continuació, utilitza el model per generar el conjunt de dades corresponent. Un cop els punts han estat creats, aquests es transfereixen a la vista per a la seva representació gràfica.

Quan l'usuari sollicita el càlcul de distàncies, el controlador selecciona l'algorisme corresponent en funció de l'opció triada. Aquest pot ser l'algorisme de força bruta, el basat en Divideix i Venceràs o el de la parella més llunyana. Un cop executat l'algorisme, el resultat es retorna en forma d'un objecte `Resultat`, que conté la informació necessària per ser representada.

Un aspecte especialment rellevant del controlador és la mesura del temps d'execució dels algorismes. Aquesta es realitza mitjançant la funció `System.nanoTime()`, permetent obtenir mesures precises en mil·lisegons. Aquesta informació es mostra a la interfície, facilitant la comparació entre diferents estratègies.

Durant el desenvolupament, una de les decisions més importants ha estat mantenir el controlador completament lliure de lògica gràfica i de càlcul complex. El seu únic objectiu és coordinar les operacions, delegant la generació de dades al model i la representació a la vista. Aquesta separació permet una arquitectura més neta i facilita tant el manteniment com l'escalabilitat del sistema.

En conjunt, el controlador actua com el pont que connecta totes les parts del sistema, assegurant un flux coherent i eficient entre la interacció de l'usuari i el càlcul dels resultats.

Per evitar el bloqueig de la interfície gràfica durant execucions amb grans volums de dades, s'ha implementat l'execució dels algorismes en segon pla mitjançant la classe `SwingWorker`. Aquesta decisió permet mantenir la interfície reactiva mentre es duen a terme càlculs intensius.

De forma complementària, s'ha incorporat un temporitzador (`Timer`) que actualitza periòdicament el temps d'execució mostrat a l'usuari, oferint una retroalimentació contínua del progrés del càlcul.

6 Algorismes Utilitzats

El problema de la cerca de la parella de punts més propera és un dels casos clàssics dins de la geometria computacional, ja que posa en evidència la importància de dissenyar algorismes eficients per gestionar grans conjunts de dades. En aquesta pràctica s'han implementat diferents estratègies amb l'objectiu de comparar-ne el comportament i analitzar el seu rendiment.

6.1 Algorisme de força bruta

La primera aproximació implementada és l'algorisme de força bruta, que constitueix la solució més directa i intuïtiva al problema. Aquest mètode consisteix en comparar totes les possibles parelles de punts dins del conjunt, calculant la distància entre cada parella i mantenint la mínima trobada.

A nivell d'implementació, aquesta estratègia es basa en dos bucles imbricats que recorren la llista de punts. Per a un conjunt de n punts, el nombre total de comparacions és de l'ordre de $\frac{n(n-1)}{2}$, fet que implica una complexitat temporal de $O(n^2)$.

Tot i la seva simplicitat i el fet que garanteix sempre la solució òptima, aquest algorisme presenta un creixement molt ràpid del temps d'execució. A mesura que el nombre de punts augmenta, el nombre de comparacions creix de manera quadràtica, fent-lo impracticable per a conjunts de dades grans.

6.2 Algorisme Divideix i Venceràs

Per tal de millorar el rendiment, s'ha implementat una estratègia basada en el paradigma de Divideix i Venceràs. Aquest algorisme parteix de la idea de reduir el

nombre de comparacions necessàries dividint el problema en subproblemes més petits.

En primer lloc, els punts es ordenen segons la seva coordenada x . A continuació, el conjunt es divideix en dues meitats, esquerra i dreta, resolent el problema de manera recursiva en cadascuna d'elles. Aquest procés es repeteix fins arribar a subconjunts petits, on es pot aplicar directament la solució de força bruta.

Un cop resolt els subproblemes, es combina la informació per obtenir la solució global. En aquest punt apareix el cas més delicat de l'algorisme: la possibilitat que la parella més propera estigui formada per punts situats en diferents meitats.

Per gestionar aquesta situació, es construeix una franja central (strip) al voltant de la línia de divisió. Aquesta franja conté els punts que es troben a una distància inferior a la millor distància trobada fins al moment. Els punts dins aquesta franja es processen ordenats per la coordenada y , i es demostra que només és necessari comparar cada punt amb un nombre constant de veïns propers (com a màxim 7).

Aquesta optimització és clau, ja que permet reduir dràsticament el nombre de comparacions i mantenir la complexitat global de l'algorisme en $O(n \log n)$. En la pràctica, aquest enfocament resulta molt més eficient que la força bruta, especialment en conjunts de dades grans.

6.3 Algorisme de la parella més llunyana

Per al càlcul de la parella de punts més llunyana s'ha optat per una estratègia basada en força bruta. Aquest algorisme segueix una lògica similar a la utilitzada per a la distància mínima, però en aquest cas es conserva la distància màxima trobada.

Tot i que existeixen solucions més eficients per aquest problema, l'ús de força bruta resulta suficient en el context d'aquesta pràctica, ja que el cost computacional es manté dins de valors acceptables per als conjunts de dades utilitzats.

6.4 Optimització mitjançant buckets

Com a millora addicional, s'ha implementat una estratègia basada en partició espacial mitjançant una graella de buckets. L'objectiu d'aquest mètode és reduir el nombre de comparacions necessàries en el càlcul de la parella de punts més propera, limitant la cerca a regions locals del pla.

La idea principal consisteix en dividir l'espai bidimensional en cel·les quadrades de mida fixa i assignar cada punt a una d'elles en funció de les seves coordenades. D'aquesta manera, es transforma el problema global en múltiples subproblemes més petits, ja que cada punt només necessita comparar-se amb un subconjunt reduït de punts.

Un dels aspectes més importants de la implementació és la determinació de la mida de les cel·les. En lloc de fixar una mida arbitrària, es realitza una estimació inicial de la distància mínima utilitzant un subconjunt reduït de punts. Aquesta distància aproximada es fa servir com a mida de cella, fet que permet adaptar la graella a la distribució real de les dades. Com que els punts es troben normalitzats en l'interval $(0, 1)$, aquesta mida determina directament el nombre de cel·les que es generen en cada eix.

Un cop definida la mida de les cel·les, cada punt es mapeja a una cella concreta calculant els índexs enters corresponents als seus valors de coordenades. Aquesta assignació permet emmagatzemar els punts en una estructura de tipus mapa, on cada clau identifica una cella i conté la llista de punts que hi pertanyen.

Un repte important d'aquesta estratègia és la possibilitat que la parella de punts més propera es trobi distribuïda entre dues cel·les diferents. Si només es comparassin els punts dins de la mateixa cella, es podrien perdre solucions òptimes situades a cavall entre regions adjacents.

Per resoldre aquest problema, la implementació amplia la cerca a les cel·les veïnes. En concret, per a cada punt es consideren no només els punts de la seva pròpia cella, sinó també els de les vuit cel·les adjacents. D'aquesta manera, es garanteix que qualsevol parella de punts separada per una distància menor que la mida de la cella serà detectada, encara que es trobi en diferents buckets.

Aquesta decisió permet mantenir la correcció de l'algorisme, assegurant que no es perden possibles solucions òptimes, al mateix temps que es redueix considerablement el nombre de comparacions en el cas mitjà. Tot i això, el rendiment del mètode depèn de la distribució dels punts, ja que una alta concentració en una mateixa cella pot incrementar el nombre de comparacions necessàries.

6.4.1 Adaptació de la graella segons el nombre de punts

La mida de les cel·les de la graella depèn directament de la distància mínima estimada entre punts. Això provoca que la graella s'adapti dinàmicament en funció del nombre de punts generats.

A la Figura 8 es pot observar el cas amb 10000 punts. En aquest cas, la distància entre punts és relativament gran, fet que dona lloc a una graella amb poques cel·les (aproximadament 4×4).

En canvi, a la Figura 9 es mostra el cas amb 1000000 punts. En aquesta situació, la densitat de punts és molt més elevada, la distància mínima disminueix i la graella es torna molt més fina, arribant aproximadament a una configuració de 60×60 cel·les.

Aquesta adaptació dinàmica és fonamental per al rendiment de l'algorisme, ja que permet reduir el nombre de comparacions limitant la cerca a regions locals

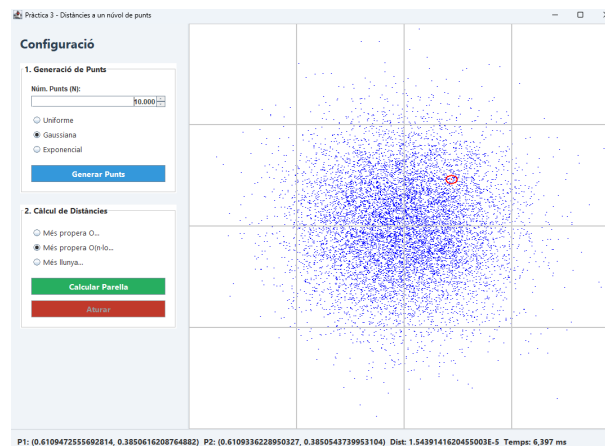


Figura 8. Graella generada amb 10000 punts (baixa densitat).

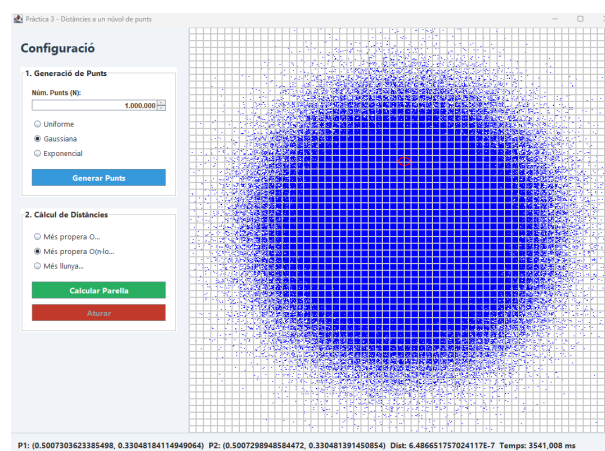


Figura 9. Graella generada amb 1000000 punts (alta densitat).

del pla, independentment de la mida del conjunt de dades.

7 Resultats Experimentals i Anàlisi de Rendiment

Per analitzar el comportament dels algorismes, s'han realitzat proves amb conjunts de punts de mida variable, utilitzant distribucions uniformes. L'objectiu ha estat comparar el rendiment de les diferents estratègies implementades i estudiar la seva escalabilitat.

7.1 Experiment amb 100000 punts

Per a un conjunt de 100000 punts, s'han obtingut els següents resultats per a l'algorisme de força bruta:

Execució	Temps (ms)
1	6963.57
2	6470.90
3	6433.65
4	6429.87
5	6468.45
6	6460.12

El temps mitjà és aproximadament de 6537 ms.

Per al mateix conjunt de dades, utilitzant l'algorisme optimitzat amb buckets:

Execució	Temps (ms)
1	149.48
2	85.01
3	117.93
4	90.13
5	84.41
6	78.69

El temps mitjà és aproximadament de 100.94 ms.

Això representa una millora de més de 60 vegades respecte a l'algorisme de força bruta.

7.2 Experiment amb 500000 punts

Per a un conjunt de 500000 punts, els resultats mostren una diferència encara més significativa.

7.2.1 Algorisme de força bruta

Execució	Temps (ms)
1	199102
2	204107
3	180623
4	179775
5	179256

El temps mitjà obtingut és aproximadament de 186518 ms (186.5 segons).

7.2.2 Algorisme optimitzat amb buckets

Execució	Temps (ms)
1	1097
2	970
3	1789
4	833
5	694

El temps mitjà és aproximadament de 1077 ms.

7.3 Escalabilitat amb 1000000 punts

Finalment, s'han realitzat proves amb un conjunt d'un milió de punts, amb l'objectiu d'analitzar el comportament dels algorismes en escenaris de gran escala.

7.3.1 Algorisme de força bruta

Només s'ha pogut realitzar una execució, ja que el cost computacional és extremadament elevat:

Execució	Temps (ms)
1	954138

Aquest temps correspon aproximadament a 954 segons (prop de 16 minuts), fet que confirma que l'algorisme de força bruta és completament inviable per a aquest volum de dades.

7.3.2 Algorisme optimitzat amb buckets

Execució	Temps (ms)
1	1934
2	5812
3	2191
4	11508
5	1817
6	3904

El temps mitjà és aproximadament de 4528 ms.

7.3.3 Comparació

Comparant ambdós resultats, es pot observar una millora superior a 200 vegades en el temps d'execució.

Tot i que es detecta una major variabilitat en els temps de l'algorisme de buckets, aquest continua mantenint un rendiment molt eficient, resolent el problema en pocs segons fins i tot amb un milió de punts.

Aquesta variabilitat es pot atribuir a factors com la distribució aleatòria dels punts, la mida de les cel·les generades i la gestió de memòria de la màquina virtual de Java.

7.4 Anàlisi global

Els resultats experimentals mostren clarament la diferència de comportament entre els algorismes. Mentre que l'algorisme de força bruta presenta un creixement quadràtic que el fa inviable per a grans volums de dades, les estratègies optimitzades mantenen temps d'execució molt reduïts.

A mesura que augmenta el nombre de punts, aquesta diferència es fa encara més evident, arribant a factors de millora superiors a dos ordres de magnitud.

En conjunt, aquests resultats confirmen la importància d'utilitzar algorismes optimitzats en problemes de geometria computacional, especialment quan es treballa amb grans conjunts de dades.

8 Conclusions

8.1 Conclusions Tècniques

El desenvolupament d'aquesta pràctica ha permès analitzar en profunditat la importància de l'eficiència algorísmica en la geometria computacional. Les principals conclusions extretes són:

- **Escalabilitat:** Mentre que l'algorisme de força bruta és funcional per a conjunts petits, el seu creixement quadràtic el fa inviable per a grans

volums de dades, arribant a tardar aproximadament 16 minuts per a un milió de punts.

- **Optimització:** L'estratègia de partició espacial mitjançant *buckets* ha demostrat ser la solució més robusta, reduint el temps d'execució a menys de 5 segons per al mateix volum de dades, aconseguint una millora superior a 200 vegades respecte al mètode base.
- **Arquitectura del Programa:** L'aplicació del patró MVC ha estat fonamental per mantenir la independència entre la lògica matemàtica i la interfície gràfica, facilitant la mantenibilitat i l'extensibilitat del sistema.

8.2 Reflexió Personal

Més enllà dels resultats numèrics, aquesta pràctica ens ha aportat una visió més profunda sobre com afrontar problemes reals en l'enginyeria informàtica.

Durant el procés, el repte més gran no ha estat només implementar la lògica matemàtica, sinó gestionar la **interactivitat de l'usuari**. Hem après que, en aplicacions que manegen milions de dades, no n'hi ha prou amb tenir un algorisme ràpid; cal garantir que la interfície sigui reactiva. La implementació de l'execució en segon pla mitjançant fils i la incorporació del **botó d'aturada** han estat decisions clau per millorar l'experiència d'ús, permetent a l'usuari mantenir el control sobre processos pesats en tot moment.

Passar de la teoria de la complexitat algorísmica a veure com un canvi en l'estructura de dades pot estalviar minuts de processament ha estat una experiència molt gratificant. Aquesta pràctica ens ha servit per consolidar la idea que l'optimització no és un luxe, sinó una necessitat quan treballem amb dades massives, i ens ha preparat millor per dissenyar sistemes eficients i ben estructurats en el futur.

Referències

- [1] Assignatura d'Algorismes Avançats, Universitat de les Illes Balears, 2026.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [3] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Springer, 2008.
- [4] *ChatGPT*, model GPT-5.3, OpenAI.